
Authentication Mechanisms and Protocols

Authentication (Authn)

- ▷ Proof that an entity has an identity attribute it claims to have
 - ♦ I'm **Joe**
 - ♦ Prove it!
 - ♦ Here is my **proof**,
calculated with **Joe's credentials** that I've agreed with you
 - ♦ Proof accepted/not accepted

- ♦ I'm **over 18**
- ♦ Prove it!
- ♦ Here is a **claim** issued by a competent authority,
which I can also **prove that I'm the owner**
- ♦ **Proof and claim** accepted/not accepted

Authn: Proof Types

- ▷ Something we know
 - ◆ A secret memorized (or written down...) by Joe
 - ▷ Something we have
 - ◆ An object/token solely held by Joe
 - ▷ Something we are
 - ◆ Joe's Biometry
- ▷ Multi-factor authentication (MFA)
 - ◆ Simultaneous use of different proof types
 - ◆ 2FA = Two Factor Authentication
 - ▷ Risk-based MFA
 - ◆ Variable MFA
 - ◆ Higher attack risk, more factors or less risky factors
 - ◆ Lower attack risk, less or easier factors

Authn : Goals

- ▷ Authenticate interactors
 - ◆ People, services, servers, hosts, networks, etc.
- ▷ Enable the enforcement of authorization policies and mechanisms
 - ◆ Authorization $\neq$ authentication
 - ◆ Authorization $\Rightarrow$ authentication
- ▷ Facilitate the exploitation of other security-related protocols
 - ◆ e.g. key distribution for secure communication

Authn : Requirements

▷ Trustworthiness

- ◆ How good is it in proving the identity of an entity?
- ◆ How difficult is it to be deceived?
- ◆ Level of Assurance (LoA)

▷ Secrecy

- ◆ No disclosure of secret credentials used by legit entities

NIST 800-63

LoA	DESCRIPTION	TECHNICAL REQUIREMENTS		
		IDENTITY PROOFING REQUIREMENTS	TOKEN (SECRET) REQUIREMENTS	AUTHENTICATION PROTECTION MECHANISMS REQUIREMENTS
1	Little or no confidence exists in the asserted identity; usually self-asserted; essentially a persistent identifier	Requires no identity proofing	Allows any type of token including a simple PIN	Little effort to protect session from off-line attacks or eavesdropper is required.
2	Confidence exists that the asserted identity is accurate; used frequently for self service applications	Requires some identity proofing	Allows single-factor authentication. Passwords are the norm at this level.	On-line guessing, replay and eavesdropping attacks are prevented using FIPS 140-2 approved cryptographic techniques.
3	High confidence in the asserted identity's accuracy; used to access restricted data	Requires stringent identity proofing	Multi-factor authentication, typically a password or biometric factor used in combination with a 1) software token, 2) hardware token, or 3) one-time password device token	On-line guessing, replay, eavesdropper, impersonation and man-in-the-middle attack are prevented. Cryptography must be validated at FIPS 140-2 Level 1 overall with Level 2 validation for physical security.
4	Very high confidence in the asserted identity's accuracy; used to access highly restricted data.	Requires in-person registration	Multi-factor authentication with a hardware crypto token.	On-line guessing, replay, eavesdropper, impersonation, man-in-the-middle, and session hijacking attacks are prevented. Cryptography in the hardware token must be validated at FIPS 140-2 level 2 overall, with level 3 validation for physical security.

Authn : Requirements

▷ Robustness

- ◆ Prevent attacks to the protocol data exchanges
- ◆ Prevent on-line DoS attack scenarios
- ◆ Prevent off-line dictionary attacks

▷ Simplicity

- ◆ It should be as simple as possible to prevent entities from choosing dangerous shortcuts

▷ Deal with vulnerabilities introduced by people

- ◆ They have a natural tendency to facilitate or to take shortcuts
- ◆ Deal with phishing!

Authn: Entities and deployment model

Entities

- ▷ People
- ▷ Hosts
- ▷ Networks
- ▷ Services / servers

Deployment model

- ▷ Along the time
 - ♦ Only when interaction starts
 - ♦ Continuously along the interaction
- ▷ Directionality
 - ♦ Unidirectional
 - ♦ Bidirectional (mutual)

Authn interactions: Basic approaches

▷ Direct approach

- ◆ Provide credentials
- ◆ Wait for verdict
- ◆ Advantage: no computations by the presenter
- ◆ Disadvantage: credentials can be exposed to malicious validators

▷ Challenge-response approach

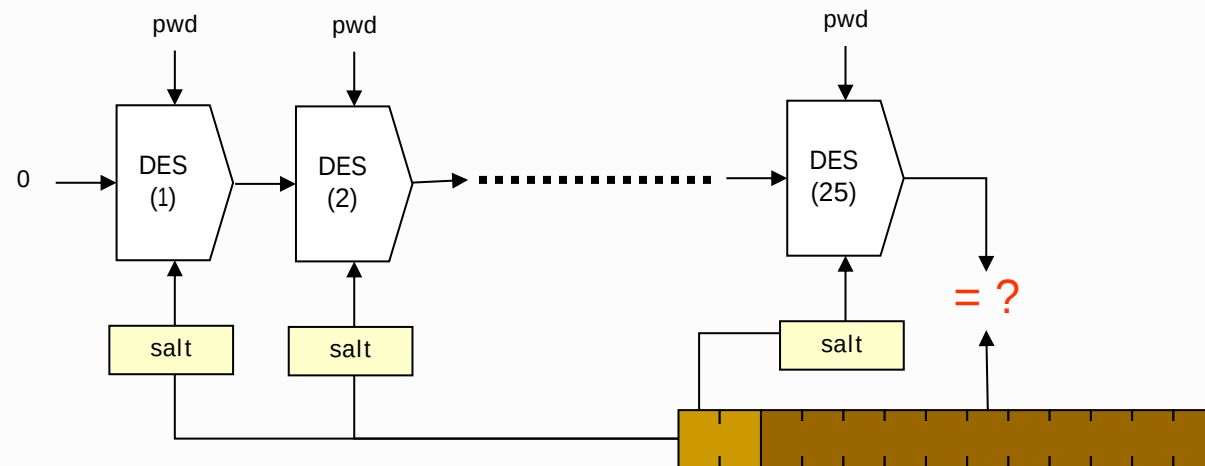
- ◆ Get challenge
- ◆ Provide a response computed from the challenge and the credentials
- ◆ Wait for verdict
- ◆ Advantage: credentials are not exposed to malicious validators
- ◆ Disadvantage: requires computations by the presenter

Authn of subjects:

Direct approach w/ known password

- ▷ A password is checked against a value previously stored
 - ♦ For a claimed identity (username)
- ▷ Personal stored value:
 - ♦ Transformed by a unidirectional function
 - ♦ Windows: digest function
 - ♦ UNIX: DES hash + salt
 - ♦ Linux: MD5 + salt
 - hash is configurable
- ▷ Optimal: PBKDF2 or script, with high complexity

Authn of subjects: Direct approach w/ known password



$\text{DES hash} = \text{DES}_{\text{pwd}}^{25}(0)$

$\text{DES}_k^n(x) = \text{DES}_k(\text{DES}_k^{n-1}(x))$

Permutation of 12 subkeys' bit pairs with salt (12 bits)

Authn of subjects:

Direct approach w/ known password

▷ Advantage

- ♦ Simplicity!
- ♦ (Useful) impersonation

▷ Problems

- ♦ Malicious impersonation
 - Phishing!
- ♦ Usage of weak passwords
 - Enable dictionary attacks
- ♦ Transmission of passwords along insecure communication channels
 - Eavesdroppers can easily learn the password
 - e.g. Unix remote services, PAP



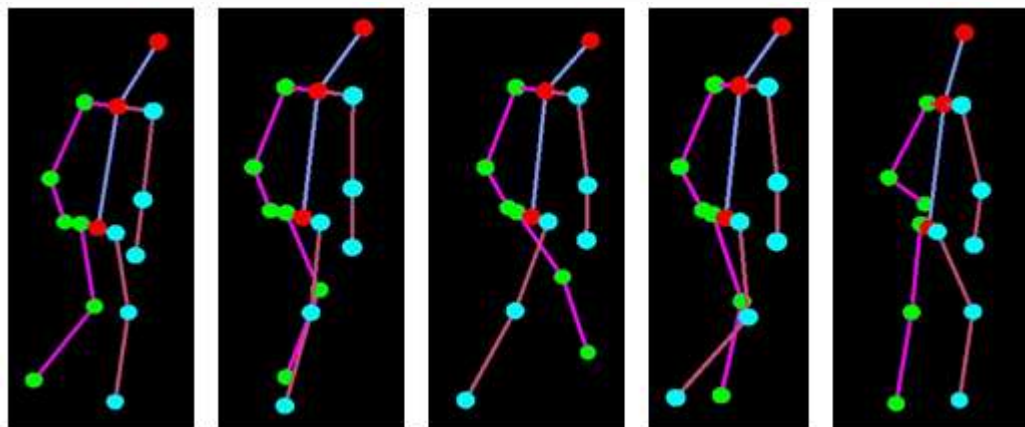
Top Ten 2017 from Splashdata

1. 123456
2. Password
3. 12345678
4. qwerty
5. 12345
6. 123456789
7. letmein
8. 1234567
9. football
10. iloveyou

Authn of people:

Direct approach with biometrics

- ▷ People get authenticated using body measures
 - ◆ Biometric samples
 - ◆ Fingerprint, iris, face geometry, voice timber, manual writing, vein matching, etc.
- ▷ Measures are compared with personal records
 - ◆ Biometric references (or template)
 - ◆ Registered in the system with a previous enrolment procedure
- ▷ Identification vs authentication
 - ◆ Identification: 1-to-many check for a match
 - ◆ Authentication: 1-to-1 check for a match w/ a claimed template



Authn of people:

Advantages of biometrics

- ▷ People do not need to use memory, or carry something
 - ♦ Just be themselves
- ▷ People cannot choose weak passwords
 - ♦ In fact, they don't choose anything
- ▷ Authentication credentials cannot be transferred to others
 - ♦ One cannot delegate their own authentication

Authentication of people:

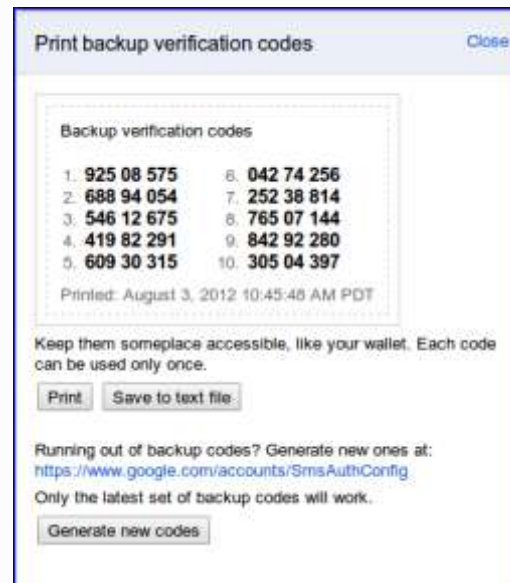
Problems with biometrics

- ▷ Biometric methods are still incipient
 - ◆ In many cases it can be fooled with ease (Face Recognition, Fingerprint)
- ▷ People cannot change credentials
 - ◆ If the credentials or templates are stolen
- ▷ Credentials cannot be transferred between individuals
 - ◆ If it is required in extraordinary scenarios
- ▷ Can pose risks to individuals
 - ◆ Physical integrity can be compromised by an attacker in order to acquire biometric data
- ▷ It is not easy to be implemented in remote systems
 - ◆ It is mandatory to have secure and trusted biometric acquisition devices
- ▷ Biometrics can reveal other personal secrets
 - ◆ Diseases

Authn of subjects:

Direct approach with one-time passwords

- ▶ One-Time Passwords = secrets that cannot be reused
 - ◆ Pre-distributed directly, or the result of a generator function
- ▶ Example: Bank codes, Google Backup Codes



https://www.montepio.pt/SitePublico/pt_PT/particulares/montepio24/cartao-matriz.page?altcode=10006P

Authn of subjects:

Direct approach with one-time passwords

▷ Advantages

- ♦ Can be eavesdropped, allowing its use in channels without encryption
- ♦ Can be chosen by the authenticator, which may enforce a given complexity
- ♦ Can depend on a shared password

▷ Problems

- ♦ Interacting entities need to know which password to use on each occasion
 - Implies some form of synchronization (e.g., index, coordinates)
- ♦ Individuals may require additional resources to store/generate the passwords
 - Sheet of paper, application, additional device, etc.

Authn of subjects: OTPs from a shared key

- ▷ HOTP (Hash-based One Time Password, RFC 4226)
 - ◆ OTP generated from a counter and a shared key
 - ◆ Counters are updated independently

- ▷ TOTP (Time-based One Time Password, RFC 6238)
 - ◆ OTP generated from a timestamp and a shared password
 - ◆ TOTP is HOTP with timestamps instead of counters
 - ◆ Clocks need a rough synchronization

HOTP (HMAC-based one-time password, RFC 4226)

- ▷ Numeric OTP computed from shared key K and synchronized counter C
 - ♦ Hash key and counter
 - And increase counter
 - ♦ From hash, get a (floating) portion of 31 contiguous bits
 - Dynamic Binary Code (DBC)
 - ♦ Compute a d -long decimal number
 - $d \geq 6$
- ▷ Issues
 - ♦ Counter synchronization upon a failure
 - If the authenticator keeps it after a failure, exhaustive search attacks are viable
 - If the authenticator always increments it, DoS attacks are possible
 - ♦ Acceptance windows
 - Mitigates minor desynchronizations, but decreases security

TOPT (Time-based one-time password, RFC 6238)

▷ HOTP with a counter derived from time

▷ $C_T = \left\lfloor \frac{T - T_0}{T_x} \right\rfloor$

- ♦ T – initial time

- ♦ T_0 – initial time

- ♦ T_x – time interval (default: 30 seconds)

▷ $\text{TOTP}(K) = \text{HOTP}(K, C_T)$

Challenge-response Approach

- ▷ The authenticator provides a challenge
 - ◆ A nonce (value not once used)
 - ◆ Usually random (simpler approach if large, hundreds of bits)
 - ◆ Can be a counter
- ▷ The authenticated entity transforms the challenge
 - ◆ The transformation method is agreed with the authenticator
- ▷ The result is sent to the authenticator
- ▷ The authenticator verifies the result
 - ◆ Calculates a result using the same method and challenge
 - ◆ Or produces a value from the result and evaluates if it is equal to the challenge, or to some related value

Challenge-Response Approach

▷ Advantages

- ♦ Authentication credentials are not exposed
- ♦ An eavesdropper will see the challenge and the result
 - but has no knowledge about the transformation

▷ Problems

- ♦ Authenticated entities must have the capability of calculating responses
 - Hardware token ou software application
- ♦ The authenticator may need to keep shared secrets (in clear text)
 - Secrets can be stolen
 - Individuals may reuse secrets in other systems, enabling lateral attacks
- ♦ May be possible to calculate all results to a single (or all) challenge(s)
 - Can reveal the secret used
- ♦ May be vulnerable to dictionary attacks
- ♦ Authenticator should NEVER issue the same challenge to the same user

Authn of Subjects:

Challenge-Response with key pairs

▷ Authentication Credentials

- ♦ A key pair, possibly stored in a token
 - e.g., the Citizen Card
- ♦ The private key computes responses
 - These are signatures
- ♦ Some secret/action may allow the signature
 - PIN, presence

▷ The authenticator knows

- ♦ The user public key

▷ Robust against:

- ♦ Dictionary attacks
- ♦ Offline attacks to the database
- ♦ Insecure channels



Authn of Subjects: Challenge-Response with key pairs

▷ Protocol

- ♦ The authenticator generates a challenge
- ♦ Key pair owner signs the challenge
 - With their private key
- ♦ The authenticator validates the signature with the subject public key
 - If it is a valid challenge signature, the authentication is successful

▷ Encryption is also possible

- ♦ Though not common

Authn of Subjects: Challenge-Response with FIDO 2

▷ FIDO2 tokens (FIDO Alliance)

- ♦ For both mobile and desktop environments
- ♦ Web Authentication (WebAuthn) specification
- ♦ Client-to-Authenticator Protocol (CTAP)
- ♦ Security
 - Credentials never leave the user's device and are never stored on a server
 - No risks of phishing, no password theft (still, tokens can be stolen)
 - No replay attacks
 - Token certification levels
- ♦ Privacy
 - Credentials are unique per website
 - Tracking is not possible (different web sites, different public keys for the same token)
 - Biometric data, when used, never leaves the user's device



FIDO2 certification

FIDO Authenticator Certification Examples

L3+



USB U2F Token built on a CC-certified Secure Element **Certification: L3+**

L3



USB U2F Token built on a basic simple CPU, OS, is certified. Good physical anti-tampering enclosure



UAF implemented as a TA running on a certified TEE with POP memory

L2



UAF implemented as a TA in an uncertified TEE

L1+



UAF in downloadable app using white box crypto and other techniques
Certification: L1+

L1



Downloaded app making use of Touch ID on iOS
Certification: L1



FIDO2 making use of the Android keystore. Keystore is not certified
Certification: L1



FIDO2 built into a downloadable web browser app
Certification: L1

Authn of Subjects:

Challenge-Response with shared secret

▷ Authentication credentials

- ◆ Password selected by the individual

▷ The authenticator knows:

- ◆ Bad approach: the shared password
- ◆ Better approach: a (slow) transformation of the shared password
 - The transformation should be unidirectional

Authn of Subjects:

Challenge-Response with shared secret

▷ Protocol

- ♦ The authenticator generates a challenge
- ♦ The individual calculates a transformation of the challenge and the password
 - $\text{result} = \text{hash}(\text{challenge} \parallel \text{password})$
 - or... $\text{result} = \text{encrypt}(\text{challenge}, \text{password})$
- ♦ The authenticator reverts the process and checks if the values match
 - $\text{result} == \text{hash}(\text{challenge} \parallel \text{password})$
 - or $\text{challenge} == \text{decrypt}(\text{result}, \text{password})$
- ♦ Examples with shared passwords: CHAP, MS-CHAP v1/v2, S/Key
- ♦ Examples with shared keys: SIM & USIM (celular communications)

PAP and CHAP

(RFC 1334, 1992, RFC 1994, 1996)

- ▷ Protocols user for PPP (Point-to-Point Protocol)
 - ◆ **Unidirectional authentication**
 - The authenticator authenticates users, but users do not authenticate the authenticator
- ▷ PAP (PPP Authentication Protocol)
 - ◆ Simple presentation of a UID/password pair
 - ◆ Insecure transmission (in clear text)
- ▷ CHAP (CHallenge-response Authentication Protocol)
 - Aut → U : authID, challenge
 - U → Aut: authID, MD5(authID, secret, challenge), identity
 - Aut → U : authID, OK/not OK
 - ◆ The authenticator can request further authentication at any time

Authn of subjects:

Challenge-Response with shared key

- ▷ Uses a cryptographic key instead of a password
 - ♦ Robust against dictionary attacks
 - ♦ Requires a device to store the shared key

GSM Subscriber authentication

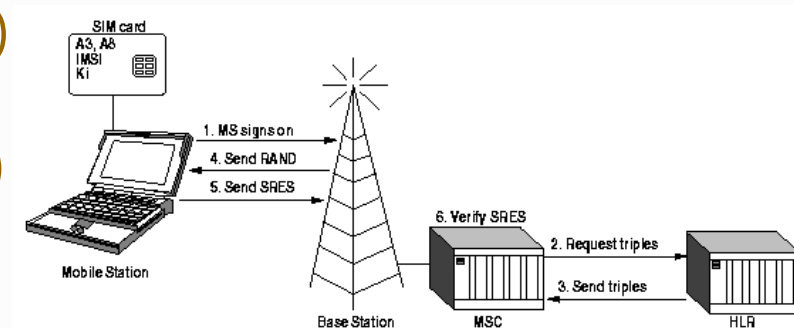
- ▷ Uses a secret shared between the HLR and the subscriber phone
 - ◆ Uses 128-bit shared key (not an asymmetric key pair)
 - ◆ Key is stored in the SIM card
 - ◆ SIM card is unlocked by a user PIN
 - ◆ SIM card answers challenges using the shared key

- ▷ Uses (initially unknown algorithms):
 - ◆ A3 for authentication
 - ◆ A8 to generate the session key
 - ◆ A5 is a stream cipher for communication

- ▷ A3 and A8 executed by the SIM, A5 executed by the baseband
 - ◆ A3 and A8 can be chosen by the operator

GSM Subscriber authentication

- ▷ MSC requests triples from HLR/AUC
 - ♦ RAND, SRES, Kc
 - ♦ It can ask one or several
- ▷ HLR generates RAND and the triples using the subscriber Ki
 - ♦ RAND, random value (128 bits)
 - ♦ SRES = A3 (Ki, RAND) (32 bits)
 - ♦ Kc = A8 (Ki, RAND) (64 bits)



- ▷ Frequently uses COMP128 for the A3/A8 algorithms
 - ♦ Recommended by the GSM consortium
 - ♦ [SRES, Kc] = COMP128 (Ki, RAND)

Authentication of systems

- ▷ By name (DNS) or MAC/IP address
 - ♦ Extremely weak, without cryptographic proof
 - Still... it is used by some services
 - e.g., NFS, TCP wrappers
- ▷ With cryptographic keys
 - ♦ Secret keys, shared between entities that communicate frequently
 - ♦ Asymmetric key pairs, one per host
 - Public keys pre-shared with entities that communicate frequently
 - Public keys certified by a third party (a CA)

Authentication of services

- ▷ Authentication of the host
 - ♦ All services co-located in the same host are automatically and indirectly authenticated
- ▷ Credentials exclusive to each service
- ▷ Authentication:
 - ♦ Secret keys shared with clients
 - When they require authentication of the clients
 - ♦ Asymmetric key pairs by host/service
 - Certified by others or not

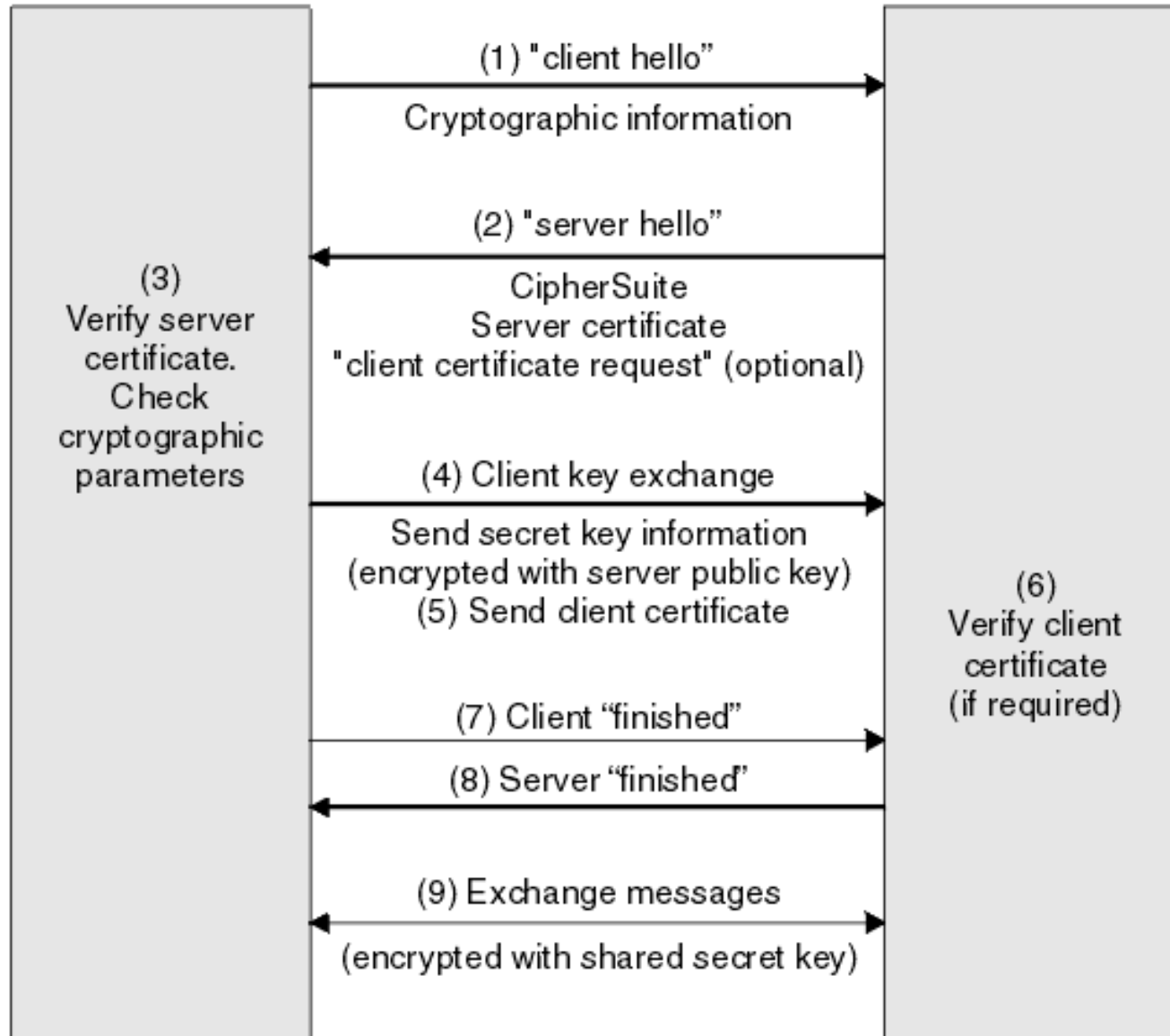
TLS

(Transport Layer Security, RFC 2246)

- ▷ Secure communication protocol over TCP/IP
 - ◆ Evolved from the SSL V3 (Secure Sockets Layer) standard
 - ◆ Manages secure sessions over TCP/IP, individual to each application
 - Initially designed for HTTP traffic
 - Currently used for many other types of traffic
- ▷ Security mechanisms
 - ◆ Negotiation of ciphers, digests and other mechanisms
 - ◆ Session key agreement
 - ◆ Confidentiality and integrity of the communication
 - ◆ Authentication of the intervenient
 - Servers, services, etc...
 - Clients (not so common)
 - Both with key pairs and X.509 certificates

SSL Client

SSL Server



TLS Ciphersuites

- ▷ If a server supports a single algorithm, it cannot expect all clients to also support it
 - ♦ More powerful/limited, older/newer
- ▷ The ciphersuite concept allows the negotiation of mechanisms between client and server
 - ♦ Both send their supported ciphersuites, and select one they both share
 - ♦ The server chooses
- ▷ Example: ECDHE-RSA-AES128-GCM-SHA256
 - ♦ Key agreement algorithm: ECDHE (Elliptic Curve Diffie-Hellman w/ Ephemeral key pairs)
 - ♦ Authentication algorithm: RSA
 - ♦ Cipher algorithm and cipher mode: AES-128 GCM
 - ♦ Hashing algorithm: SHA256

SSH (Secure SHell)

- ▷ Manages secure console sessions over TCP/IP
 - ◆ Initially designed to replace the Telnet application/protocol
 - ◆ Currently used in many other applications
 - Execution of remote commands in a secure manner (rsh/rexec)
 - Secure copy of contents from/to remote hosts (rcp)
 - Secure FTP (sftp)
 - Secure (Generic) communication tunnels (carry standard IP packets)
- ▷ Security Mechanisms
 - ◆ Session key agreement
 - ◆ Confidentiality and integrity of the communications
 - ◆ Authentication of the intervenient
 - Server / Hosts
 - Client users
 - Both achieved through several, and differentiated mechanisms

SSH: Authentication mechanisms

- ▷ Server: a pair of asymmetric keys
 - ◆ Public keys are distributed during the interaction
 - Not certified!
 - ◆ Clients store the public keys from previous interactions
 - Key should be stored in some trusted environment
 - If the key changes the client is warned
 - e.g., server is reinstalled, key is regenerated, an attacker is hijacking the connection
 - Client can refuse to continue with the authentication process
 - ◆ Proof is a signature
- ▷ Clients: authentication is configurable
 - ◆ Default: username and password
 - ◆ Other: username + private key
 - The public key MUST be pre-installed in the server
 - ◆ Other: integration with PAM for alternative authentication mechanisms

Centralized network authentication

- ▷ Used for restricting network access to known clients
 - ◆ In cabled networks
 - ◆ In wireless networks
 - ◆ In VPNs (Virtual Private Networks)

- ▷ Usually implemented by a central service
 - ◆ AAA server
 - Authentication, Authorization and Accounting
 - e.g. RADIUS and DIAMETER
 - ◆ This server defines which network services the user can make use of

Authentication by an IdP

- ▷ Unique, centralized authentication for a set of federated services
 - ◆ The identity of a client, upon authentication, is given to all federated services
 - ◆ The identity attributes given to each service may vary
 - ◆ The authenticator is called Identity Provider (IdP)
 - ◆ The federated service is called a Relying Party (RP)
 - ◆ In some cases, the provided identity attributes are shown to the client

- ▷ Examples
 - ◆ Authentication at UA
 - Performed by a central, institutional IdP (idp.ua.pt)
 - The identity attributes are securely conveyed to the service accessed by the user
 - ◆ Autenticação.gov (www.autenticacao.gov.pt)
 - Performed by a central, national IdP
 - The identity attributes are shown to the user
 - ◆ Other:
 - Services used worldwide: Google, Facebook, etc.

Centralized authentication

▷ Advantages:

- ♦ Can reuse same credentials over multiple systems/services
- ♦ Single secure repository for credentials
 - More difficult to steal credentials when used in many services
- ♦ Can implement restrictions to services/systems

▷ Disadvantages:

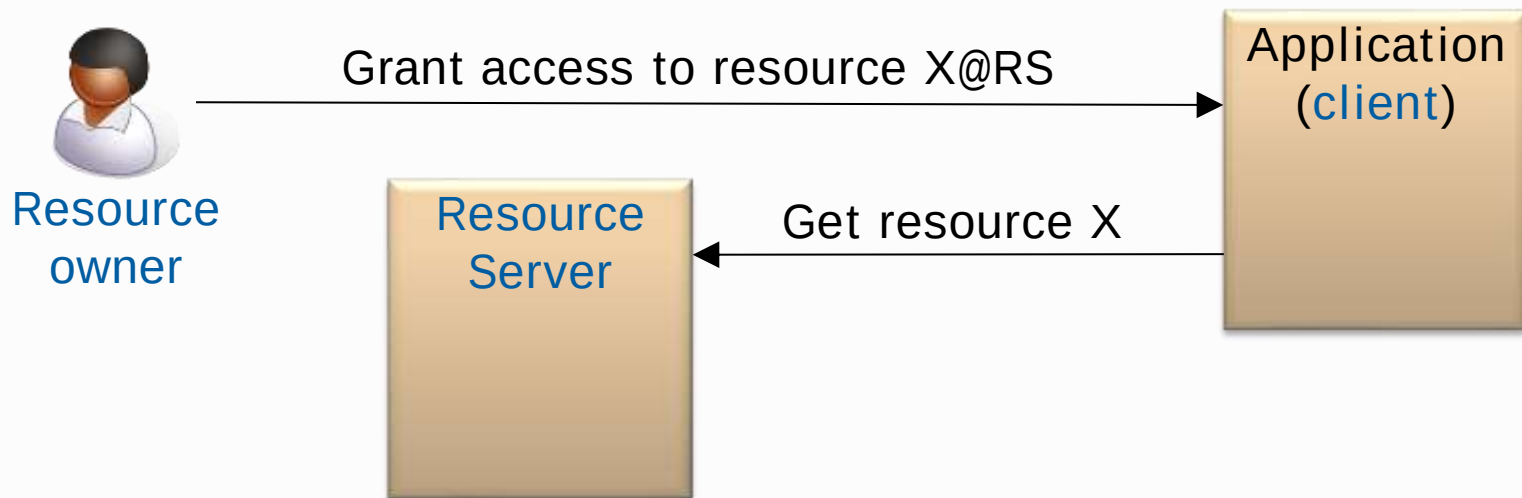
- ♦ Requires additional servers
- ♦ Single point of failure: without authentication systems, no one will be authenticated
 - Important to also deploy local credentials for admins
- ♦ Introduces delays in the authentication process
- ♦ Privacy!
 - Authenticator can track the services accessed by people

Single Sign-On

- ▷ A facility usually associated with IdP
 - ♦ Both not mandatory nor always appropriate
- ▷ SSO exists for simplifying users' life
 - ♦ They log in just once for accessing several federated services during a given time period

OAuth 2.0: delegation (RFC 6749)

- ▶ A framework to allow users to delegate access to their resources on their behalf



OAuth 2.0 roles

▷ Resource owner

- ♦ An entity capable of granting access to a protected resource
- ♦ End-user: a resource owner that is a person

▷ Resource Server

- ♦ The server hosting protected resources
- ♦ Responds to protected resource requests using access tokens

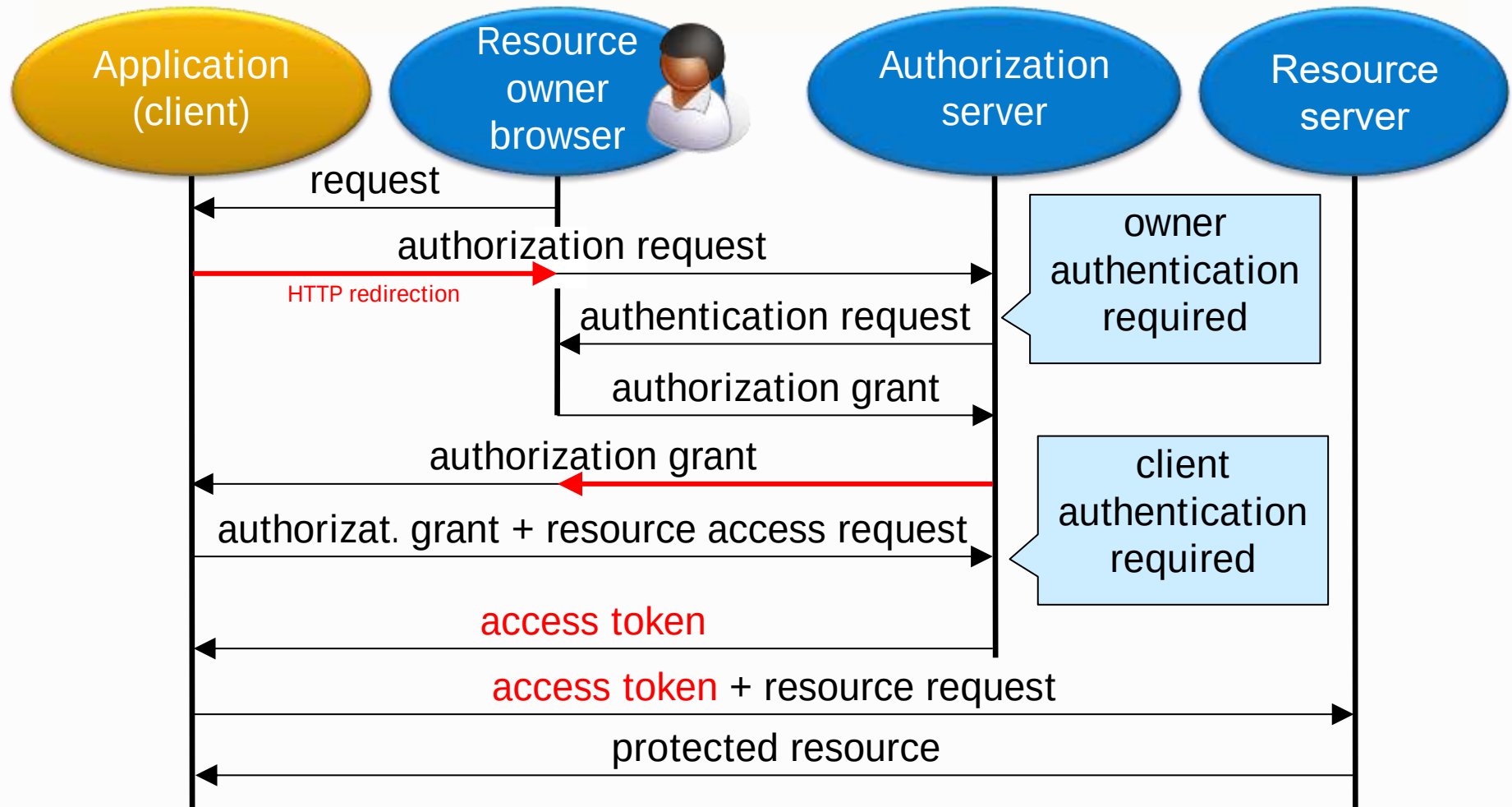
▷ Client

- ♦ An application making requests for protected resources on behalf of the resource owner and with its authorization

▷ Authorization Server

- ♦ The server issuing access tokens to clients after successfully authenticating resource owners and obtaining their authorization for the clients to access one of their (users) resources

Protocol flow



OpenID Connect (OIDC)

- ▶ An identification layer on top of OAuth 2.0
 - ♦ OAuth 2.0 provides the fundamental centralized authentication
 - ♦ The protected resources are identity attributes
 - Packed in scopes
 - The attributes are called (identity) claims